

React + TypeScript

Beginner to Confident

PART 1: Prerequisites → Foundations → Core Concepts

A self-study textbook, workbook, and roadmap combined —
built to take you from zero to confidently building typed React components.

Prepared for Gelan's Full Stack AI Engineer learning path

Table of Contents

Table of Contents	2
How to Use This Course	3
The Complete Course Roadmap	4
Level 0 — Prerequisites	5
Lesson 0.1 — JavaScript Refresher for React	5
Level 1 — Foundations	9
Lesson 1.1 — What is React and Why It Exists	9
Lesson 1.2 — JSX	13
Lesson 1.3 — Components & Props	17
Lesson 1.4 — State & Events	21
Knowledge Checkpoint — Level 1	25
Level 2 — Core Concepts (TypeScript)	26
Lesson 2.1 — Why TypeScript / Basic Types	26
Lesson 2.2 — Typing Props & State	30
Review Session — Levels 0 through 2	34
Answer Key (Levels 0–2)	35
Glossary (Levels 0–2)	38
Cheat Sheet — Levels 0–2	39
What's Next	40

How to Use This Course

This is not a book to read passively from start to finish. It's a system. Reading about React without practicing it is like reading about swimming without getting in the water — you'll recognize the words but freeze the first time you actually need them.

The Learning Loop

Every lesson is designed around this cycle. Don't skip steps, even when you're tempted to.

Learn	Read the explanation once, slowly. Don't take notes yet — just understand.
Recall	Close the lesson. Try to say the idea out loud in your own words.
Practice	Do the code example yourself, by typing it — not copy-pasting.
Explain	Answer the 'Can You Explain It?' prompts at the end of the lesson.
Review	In the Review Sessions, test yourself on older lessons, not just the newest one.
Apply	Use the concept in the mini-challenges and projects without looking back.

Tip

When you see a **STOP READING** box, actually stop. Go write the code or answer the question before continuing. The course is designed to fail as a passive read — it only works if you do the reps.

Reading the Icons

Important	A core idea — don't move on until this clicks.
Tip	A shortcut, habit, or way of thinking that saves pain later.
Common Mistake	A trap beginners fall into, and why it happens.
Think About This	A question with no single right answer — reason it through.
Practice	Something to actually type and run, not just read.
Goal	What you should be able to do by the end of a section.
Challenge	Harder — attempt it with minimal guidance.

The Complete Course Roadmap

React + TypeScript doesn't split neatly into React lessons and TypeScript lessons — TypeScript is layered on top of React from Level 2 onward, the way you'll actually use it on the job. Here is the full plan.

Part 1 (this document) covers Levels 0–2. Later parts build on top of it.

Level	What it covers	Where
Level 0 — Prerequisites	The JavaScript you need before React makes sense.	Part 1
Level 1 — Foundations	JSX, components, props, state, events — the vocabulary of React.	Part 1
Level 2 — Core Concepts	Why TypeScript exists, basic types, typing props and state.	Part 1
Level 3 — Practical Usage	Lists, conditional rendering, forms, effects, fetching data.	Part 2
Level 4 — Problem Solving	Debugging exercises and realistic beginner-to-intermediate tasks.	Part 2
Level 5 — Intermediate Concepts	Custom hooks, context, generics, advanced typing patterns.	Part 3
Level 6 — Mini Projects	Three build-from-scratch projects combining everything so far.	Part 3
Level 7 — Final Challenge	A capstone project + full exam with a separate answer key.	Part 3

Progress Tracker

Rate your confidence honestly after each lesson: 1 = I don't understand it, 5 = I could teach it.

#	Topic	Done	Confidence (1-5)
0.1	JavaScript Refresher		—
1.1	What is React & Why It Exists		—
1.2	JSX		—
1.3	Components & Props		—
1.4	State & Events		—
2.1	Why TypeScript / Basic Types		—
2.2	Typing Props & State		—

Level 0 — Prerequisites

React is a JavaScript library. If the JavaScript underneath feels shaky, React will feel like magic instead of logic. This lesson is a fast, targeted refresher — only the parts you'll lean on constantly.

LEVEL 0 · PREREQUISITES

Lesson 0.1 — JavaScript Refresher for React

1. What You Will Learn

- How to declare and use variables with **let** and **const**.
- How arrow functions work and why React code is full of them.
- How to work with arrays and objects, including destructuring.
- The basics of ES6+ syntax you'll see in almost every React file.
- Just enough about asynchronous code (Promises, `async/await`) to not be confused later.

2. Why This Matters

Every React tutorial assumes you already read JavaScript fluently. If you don't, you'll spend your mental energy decoding syntax instead of understanding React's ideas. Fixing that here means every lesson after this one can focus purely on React and TypeScript concepts.

3. The Simple Explanation

Think of JavaScript as the grammar of a language, and React as a particular writing style built with that grammar. You don't need to be a novelist to start — you just need to know enough grammar that sentences make sense when you see them.

4. Key Terms

const / let

Simple definition: Ways to create a variable.

In plain English: **const** means the variable's binding can't be reassigned. **let** means it can. React code prefers **const** almost everywhere, because most values (like a component or a piece of config) shouldn't change out from under you.

Example: `const name = "Gelan";`

Arrow function

Simple definition: A shorter way to write a function.

In plain English: Instead of writing `function add(a, b) { return a + b; }`, you write `const add = (a, b) => a + b;`. React components are almost always written this way.

Example: `const double = (n) => n * 2;`

Destructuring

Simple definition: Pulling values out of an object or array into their own variables in one line.

In plain English: Instead of writing `const name = user.name;` and `const age = user.age;` separately, you write one line that unpacks both. You will see this constantly with React props.

Example: `const { name, age } = user;`

Promise / async / await

Simple definition: A way to work with things that take time (like fetching data).

In plain English: A Promise represents a value that isn't ready yet but will be. `async/await` is a cleaner way to write code that waits for a Promise, instead of chaining `.then()` calls.

Example: `const data = await fetch(url);`

5. How It Works

A few patterns you'll see in nearly every React file, side by side with plain JavaScript:

```
// Old style function
function greet(name) {
  return "Hello, " + name;
}

// Arrow function equivalent
const greet = (name) => "Hello, " + name;

// Destructuring an object
const user = { name: "Gelan", role: "engineer" };
const { name, role } = user;

// Array methods you'll use constantly
const nums = [1, 2, 3];
const doubled = nums.map((n) => n * 2); // [2, 4, 6]
```

6. Real-World Example

Every time a React component renders a list — a table of users, a feed of posts, a shopping cart — it's using `.map()` under the hood, one of the array methods above. This single pattern shows up in almost every real app you'll build.

7. Practical Example

Walk through this step by step: turning a list of raw usernames into greeting strings.

```
const usernames = ["gelan", "maria", "leo"];

// Step 1: .map() runs the function once per item
// Step 2: whatever the function returns becomes the new array's item
const greetings = usernames.map((name) => `Hello, ${name}!`);

console.log(greetings);
// ["Hello, gelan!", "Hello, maria!", "Hello, leo!"]
```

Notice the backticks (```) — that's a **template literal**, letting you embed a variable directly inside a string with `${...}` instead of gluing strings together with `+`.

8. Common Mistakes

Common Mistake

Confusing `=` (assignment) with `===` (strict equality). Beginners often write `if (x = 5)` when they meant `if (x === 5)`. The first one *sets* `x` to 5 and is almost never what you want.

Forgetting that `.map()` returns a *new* array — it doesn't change the original one.

9. "What If?" Section

Same idea, different situations — notice what stays the same and what changes:

- What if the array is empty? `.map()` just returns an empty array — no error.
- What if you use `let` instead of `const` for `usernames`? Nothing breaks here, but React style favors `const` as a signal that you don't intend to reassign it.
- What if you need the index too? `usernames.map((name, i) => ...)` — the second parameter is always the index.

10. Check Your Understanding

- 1 What's the difference between `const` and `let`?
- 2 Rewrite this as an arrow function: `function square(x) { return x * x; }`
- 3 What does destructuring let you do, in your own words?
- 4 What does `[1, 2, 3].map(n => n + 1)` return?

5 Why does React code use `async/await` instead of raw Promises most of the time?

(Answers are in the Answer Key at the end of Level 2.)

11. Mini Challenge

Practice

Without looking back at the example: write an arrow function called `shout` that takes a string and returns it in uppercase with an exclamation mark added. Then use `.map()` to apply it to the array `["hi", "hey", "yo"]`.

12. Lesson Summary

- **const** and **let** declare variables; prefer **const** by default.
- Arrow functions are the standard function style in React code.
- Destructuring pulls values out of objects/arrays in one line — you'll use this on nearly every prop.
- `.map()` transforms every item in an array into something new — the backbone of rendering lists.
- `async/await` is how you'll write code that waits for data.

13. Can You Explain It?

Goal

Try explaining this without looking at the lesson:

- 1) What problem does destructuring solve?
- 2) Why does React rely on `.map()` so heavily?
- 3) What's the difference between a Promise and using `await` on one?

Level 1 — Foundations

This is where React itself begins. By the end of this level, you'll be able to build a small interactive component from scratch and explain every line of it.

LEVEL 1 · FOUNDATIONS

Lesson 1.1 — What is React and Why It Exists

1. What You Will Learn

- What problem React was built to solve.
- The idea of building UIs out of small, reusable pieces.
- What "declarative" means, and how it differs from what you may have done before.

2. Why This Matters

You've built things with FastAPI where you describe *what data to return* and let the framework handle the plumbing. React applies that same instinct to interfaces: you describe *what the UI should look like for a given state*, and React figures out how to update the actual page to match.

3. The Simple Explanation

Before tools like React, updating a webpage meant manually finding an element and changing it — "find the button, change its text, find the list, add an item to it." This is called **imperative** programming: step-by-step instructions for *how* to make a change.

React flips this. You write a function that describes *what the page should look like right now*, given the current data. When the data changes, you don't touch the page directly — you just describe the new version, and React works out the difference and updates only what actually changed. This is called **declarative** programming.

Think About This

Think about a thermostat. An *imperative* thermostat requires you to manually turn a dial when the room gets cold. A *declarative* thermostat lets you say "keep it at 72°" — you describe the goal, and it handles the mechanism. React is the second kind of thermostat, but for user interfaces.

4. Key Terms

Declarative UI

Simple definition: Describing what the interface should look like, not the steps to get there.

In plain English: You write "show this list of items" rather than "create an element, append it, repeat for each item."

Example: `<UserList users={users} />` — describes the outcome, not the steps.

Component

Simple definition: A reusable, self-contained piece of UI.

In plain English: A component is a JavaScript function that returns what should appear on screen. You build an app by combining many small components, the same way you'd compose small functions in Python.

Example: A Button component, a UserCard component, etc.

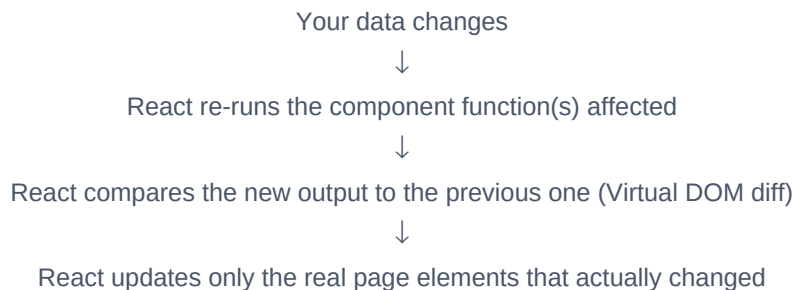
Virtual DOM

Simple definition: React's internal, lightweight copy of the real webpage structure.

In plain English: React keeps a copy of the page in memory. When data changes, it compares the new version to the old one and updates only the parts of the real page that actually changed, instead of redrawing everything.

Example: Changing one todo item's text updates just that line, not the whole list.

5. How It Works (Conceptual Flow)



6. Real-World Example

Instagram, Facebook, and Netflix's web apps are all built with React. Any interface where things update live without a full page reload — a like counter, a chat window, a shopping cart — is the kind of problem React was designed for.

7. Practical Example

Here's the smallest possible React component:

```
function Welcome() {  
  return <h1>Hello, Gelan!</h1>;  
}  
  
// This function IS a component.  
// Calling it (as <Welcome />) renders that HTML-like output.
```

That `<h1>Hello, Gelan!</h1>` isn't a string and isn't HTML — it's **JSX**, which is the entire subject of the next lesson.

8. Common Mistakes

Common Mistake

Assuming React replaces JavaScript — it doesn't. It's a library *written in* JavaScript for building UIs, the way FastAPI is a framework written in Python for building APIs.

Expecting to understand React deeply before writing any code. You'll understand the 'why' much faster once you've built even a tiny component.

9. "What If?" Section

- What if you never used React and updated the page by hand? It would work for a tiny page, but become unmanageable fast as the interface grows — this is the exact pain React exists to remove.
- What if two pieces of data change at once? React batches the updates and figures out the minimal set of real changes needed — you still just describe the end result.

10. Check Your Understanding

- 1 In your own words, what does "declarative" mean?
- 2 What is a component?
- 3 Why does React keep a Virtual DOM instead of updating the real page directly every time?
- 4 Name one real app that likely uses React and one interface behavior that suggests it.

11. Mini Challenge

Practice

Describe, in plain English (no code needed yet), what a `ProfileCard` component for a user profile might need to display. You'll build it for real in Lesson 1.3.

12. Lesson Summary

- React lets you describe UI declaratively instead of manipulating the page step by step.
- Apps are built from small, reusable components.
- The Virtual DOM lets React update only what changed, efficiently.

13. Can You Explain It?

Goal

Explain to an imaginary 12-year-old: what's the difference between telling someone exactly how to draw a picture, step by step, versus just showing them the finished picture and letting them figure out how to redraw it when something changes?

Lesson 1.2 — JSX

1. What You Will Learn

- What JSX actually is (and isn't).
- How to embed JavaScript values inside JSX.
- The rules JSX enforces that plain HTML doesn't.

2. Why This Matters

JSX is what you'll type in almost every line of every component you ever write. If its rules feel arbitrary, you'll fight the syntax constantly. Once the logic clicks, it becomes invisible.

3. The Simple Explanation

JSX looks like HTML sitting inside JavaScript, but it isn't HTML — it's **syntax sugar** (a shorthand that makes code easier to write and read) for regular JavaScript function calls. `<h1>Hi</h1>` is secretly shorthand for something like `React.createElement("h1", null, "Hi")`. You almost never need to think about that underlying call — but knowing it exists explains why JSX has its own rules.

4. Key Terms

JSX

Simple definition: A syntax extension that lets you write HTML-like markup inside JavaScript.

In plain English: It gets converted ("compiled") into plain JavaScript function calls before the browser ever sees it.

Example: `const el = <p>Hello</p>;`

Expression slot

Simple definition: A place in JSX where you can drop in a JavaScript value using curly braces.

In plain English: Anything inside `{ }` is evaluated as regular JavaScript and the result is displayed.

Example: `<p>{2 + 2}</p>` renders as "4".

Fragment

Simple definition: A wrapper that groups elements without adding an extra HTML tag to the page.

In plain English: JSX requires a single root element. If you don't want an extra `<div>` wrapping everything, use `<>...</>` instead.

Example: `<><h1>A</h1><p>B</p></>`

5. How It Works

```
const name = "Gelan";
const isLoggedIn = true;

function Greeting() {
  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>{isLoggedIn ? "Welcome back." : "Please log in."}</p>
    </div>
  );
}
```

Two rules to internalize: (1) every JSX block must return exactly **one** root element — hence wrapping the `<h1>` and `<p>` in a `<div>`. (2) Anything inside `{ }` is plain JavaScript — including ternaries, function calls, and variables, but *not* full statements like `if`.

6. Real-World Example

A weather app showing "Sunny, 24°C" pulls both words from data — JSX is what lets the markup and the live values sit together in one readable block instead of being stitched together with string concatenation.

7. Practical Example

```
const cartItems = 3;

function CartBadge() {
  return (
    <span className="badge">
      {cartItems > 0 ? `${cartItems} items` : "Cart is empty"}
    </span>
  );
}

// If cartItems is 3, this renders: <span class="badge">3 items</span>
```

8. Common Mistakes

Common Mistake

Using `class` instead of `className` — `class` is a reserved JavaScript keyword, so JSX uses `className` for the HTML `class` attribute instead.

Trying to put an `if` statement directly inside `{ }`. Curly braces only accept *expressions* (things that produce a value) — use a ternary (`cond ? a : b`) instead.

Forgetting a single root element and returning two sibling tags with nothing wrapping them.

9. "What If?" Section

- What if `isLoggedIn` were `false`? The ternary would render "Please log in." instead — same code, different output, because JSX re-evaluates the expression every time the component renders.
- What if you wanted to render nothing at all in some case? Return `null` from the expression slot — JSX simply renders nothing for it.

10. Check Your Understanding

- 1 Why can't you put a full `if` statement inside `{ }` in JSX?
- 2 Why does JSX use `className` instead of `class`?
- 3 What does a Fragment (`<>/>`) let you avoid?
- 4 What would `<p>{null}</p>` render?

11. Mini Challenge

Practice

Write a component `TempDisplay` that takes a variable `tempC = 5` and shows "Cold " if under 10, otherwise "Warm ", using a ternary.

12. Lesson Summary

- JSX is JavaScript-flavored markup that compiles down to function calls.
- Curly braces `{ }` embed JavaScript expressions, not statements.
- Every block needs one root element — use a Fragment to avoid unnecessary wrapper divs.

13. Can You Explain It?

Goal

1) What's the difference between an expression and a statement, in your own words? 2) Why does that distinction matter inside JSX curly braces?

Lesson 1.3 — Components & Props

1. What You Will Learn

- How to write a component that accepts input.
- What props are and how they flow through an app.
- Why props are read-only, and why that's a feature, not a limitation.

2. Why This Matters

Props are how components talk to each other. Almost every bug a beginner hits in their first week — "why isn't my data showing up?" — traces back to a misunderstanding of how props flow.

3. The Simple Explanation

A component that always shows the same thing is barely more useful than a static HTML file. **Props** (short for "properties") are how you pass data into a component from the outside, the same way arguments let a Python function behave differently depending on what you call it with.

Think About This

Think of a component like a function, and props like its parameters. `<UserCard name="Gelan" role="Engineer" />` is conceptually just `UserCard({name: "Gelan", role: "Engineer"})` — one object, passed in.

4. Key Terms

Props

Simple definition: Data passed into a component from its parent.

In plain English: Every component receives a single object argument. All the attributes you write in JSX (`name="..."`) become keys on that object.

Example: `<Greeting name="Gelan" />` → inside `Greeting`, `props.name` is "Gelan".

Children (props.children)

Simple definition: Whatever is nested between a component's opening and closing tags.

In plain English: It's a special, automatically-provided prop that lets a component wrap other content, like a reusable Card or Modal wrapper.

Example: `<Card><p>Inside!</p></Card>` — `props.children` is that `<p>`.

Unidirectional data flow

Simple definition: Data flows one way: from parent components down to children.

In plain English: A child can never directly change the props it receives — it can only ask its parent to change something (usually via a function passed down as a prop, covered in Lesson 1.4).

Example: A parent passes `count={5}` down; the child displays it but can't rewrite it.

5. How It Works

```
function UserCard({ name, role }) {
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <UserCard name="Gelan" role="CS Student" />
      <UserCard name="Maria" role="Designer" />
    </div>
  );
}
```

Notice `{ name, role }` in the function signature — that's the destructuring from Lesson 0.1, pulling values straight out of the props object instead of writing `props.name` everywhere.

6. Real-World Example

An e-commerce site renders one `ProductCard` component defined once, then reuses it dozens of times with different props — different name, price, and image for each product.

7. Practical Example

```
function Badge({ label, color }) {  
  return <span style={{ backgroundColor: color }}>{label}</span>;  
}  
  
// Usage:  
<Badge label="New" color="green" />  
<Badge label="Sold Out" color="red" />
```

The double curly braces in `style={{ ... }}` aren't special syntax — the outer `{ }` is the JSX expression slot, and the inner `{ }` is a plain JavaScript object literal for inline styles.

8. Common Mistakes

Common Mistake

Trying to reassign a prop inside the component, e.g. `name = "Someone else"`. Props are read-only — if a value needs to change over time, it belongs in **state** (next lesson), not props. Forgetting to pass a required prop and getting a confusing "undefined" in the rendered output instead of an obvious error.

9. "What If?" Section

- What if you don't pass a prop at all? It's undefined inside the component — no crash, just a silently missing value, which is why default values matter.
- What if you want a default value? `function Badge({ label, color = "gray" })` — destructuring supports defaults directly.
- What if a grandchild component needs a value from the top-level App? It has to be passed down through every component in between — this is called "prop drilling," and it's a real pain point you'll meet in Level 5.

10. Check Your Understanding

- 1 What are props, in your own words?
- 2 Why can't a component reassign its own props?
- 3 What is `props.children` for?
- 4 What happens if a component expects a prop that the parent never passed?

11. Mini Challenge

Practice

Build a `ProfileCard` component (the one you described back in Lesson 1.1) that accepts `name`, `title`, and `bio` as props, and render two of them with different data in an `App` component.

12. Lesson Summary

- Props pass data from parent to child components — like function arguments.
- Props are read-only inside the component that receives them.
- `props.children` carries whatever is nested inside a component's tags.
- Data flows one direction: down. Values that need to change live in state, not props.

13. Can You Explain It?

Goal

1) How are props similar to function parameters? 2) Why is it useful that a child can't modify its own props?

Lesson 1.4 — State & Events

1. What You Will Learn

- What state is and how it differs from props.
- How to use the `useState` hook.
- How to respond to user actions like clicks and typing.

2. Why This Matters

Props let data flow in, but nothing so far can change *while the app is running* in response to the user. State is what makes a page interactive instead of a static printout.

3. The Simple Explanation

State is data a component owns and can change over time — a counter, whether a menu is open, text typed into a box. When state changes, React automatically re-runs that component function and updates the page to match the new state. This is the payoff of everything in Level 1 so far.

4. Key Terms

Hook

Simple definition: A special function (always starting with "use") that lets a component tap into React features.

In plain English: Hooks are just functions you call at the top of your component — `useState` is the first and most common one.

Example: `const [count, setCount] = useState(0);`

`useState`

Simple definition: The hook that gives a component its own piece of changeable data.

In plain English: It returns a pair: the current value, and a function to update it. Calling the update function tells React "re-render this component with the new value."

Example: `const [isOpen, setIsOpen] = useState(false);`

Event handler

Simple definition: A function that runs in response to a user action.

In plain English: In JSX, you attach these with attributes like `onClick` or `onChange`, passing a function (not calling it).

Example: `<button onClick={handleClick}>`

5. How It Works

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}
```

Important

Never call `setCount( . . . )` and expect `count` to update instantly in the same line of code. State updates are scheduled — React re-runs the component on the *next* render with the new value. This trips up nearly every beginner at least once.

6. Real-World Example

A shopping cart icon that updates its number the instant you click "Add to Cart," with no page reload, is state changing and React re-rendering just that piece of the page.

7. Practical Example

```
function LoginBox() {
  const [email, setEmail] = useState("");

  function handleChange(event) {
    setEmail(event.target.value);
  }

  return (
    <div>
      <input value={email} onChange={handleChange} placeholder="Email" />
      <p>You typed: {email}</p>
    </div>
  );
}
```

This pattern — an `<input>` whose value comes from state, updated on every keystroke — is called a **controlled input**. It's the standard way React handles form fields.

8. Common Mistakes

Common Mistake

Writing `onClick={handleClick() }` instead of `onClick={handleClick}` — the first one *calls the function immediately* during render, instead of passing it to run later on click.

Directly mutating state, e.g. `count = count + 1` without calling `setCount`. React has no way to know the data changed unless you go through the setter function.

9. "What If?" Section

- What if two buttons both call `setCount` based on the old count in quick succession? You can hit state-value bugs — the safer pattern is `setCount((prev) => prev + 1)`, which always works off the latest value.
- What if state held an object instead of a number, like `{ name: "", age: 0 }`? You'd need to spread the old object when updating one field: `setUser({ ...user, age: 25 })`, since state updates replace, not merge.

10. Check Your Understanding

- 1 How is state different from props?
- 2 What does `useState(0)` return?
- 3 Why does `onClick={handleClick() }` behave differently from `onClick={handleClick}`?
- 4 Why shouldn't you mutate state directly?

11. Mini Challenge

Challenge

Build a `ToggleButton` component that shows "OFF" by default, and switches to "ON" (and back) every time it's clicked, using a single boolean piece of state. Try it with only hints, not the full solution, in mind.

12. Lesson Summary

- State is data a component owns and can change; changing it triggers a re-render.
- `useState` returns a value and a setter function — always use the setter.
- Event handlers are functions passed (not called) to attributes like `onClick`.
- Controlled inputs tie an input's value directly to state.

13. Can You Explain It?

Goal

1) Why can't you just reassign a state variable directly? 2) What does it mean that state updates are "scheduled" rather than instant?

Knowledge Checkpoint — Level 1

Before moving into TypeScript, be honest with yourself about Level 1. Answer these without notes.

- 1 Explain the difference between props and state in one or two sentences.
- 2 Write, from memory, a component with a counter that increments on click.
- 3 Why is JSX not the same thing as HTML?
- 4 What's wrong with this code, and how would you fix it? `<button onClick={setOpen(true)}>Open</button>`

Ready	You answered all four confidently and could build a small interactive component unaided.
Review Needed	You got the concepts but hesitated on state vs. props, or the event handler bug.
Not Ready	Go back to Lesson 1.3 and 1.4 and redo the Mini Challenges before continuing.

Level 2 — Core Concepts (TypeScript)

You now know enough React to build interactive components. Level 2 adds TypeScript on top — the layer that catches an entire category of bugs before your code ever runs.

LEVEL 2 · CORE CONCEPTS

Lesson 2.1 — Why TypeScript / Basic Types

1. What You Will Learn

- What problem TypeScript solves that plain JavaScript doesn't.
- The core primitive types: string, number, boolean, array, and object shapes.
- How to write a basic typed function.

2. Why This Matters

You already care about correctness — you use type hints in Python and think in terms of request/response schemas with FastAPI's Pydantic models. TypeScript brings that exact same discipline to the frontend.

3. The Simple Explanation

JavaScript won't stop you from adding a number to a string, calling a method that doesn't exist, or passing the wrong shape of data into a function — it just fails at runtime, often far from where the actual mistake was made. TypeScript adds a **type checker** that runs while you write code, catching these mistakes in your editor before you ever run the app.

Think About This

This is the same idea as a Pydantic model in FastAPI catching a malformed request before your endpoint logic runs — except TypeScript checks your own code against itself, continuously, as you type.

4. Key Terms

Type

Simple definition: A label describing what kind of value something is.

In plain English: Telling TypeScript "this variable will always be a string" lets it warn you the moment you try to treat it like a number, or vice versa.

Example: `let age: number = 25;`

Type inference

Simple definition: TypeScript guessing the type automatically, without you writing it.

In plain English: You don't always need to annotate everything — if you write `const age = 25;`, TypeScript already knows `age` is a number.

Example: `const name = "Gelan";` — inferred as `string`.

Interface / type alias

Simple definition: A named description of an object's shape.

In plain English: Instead of repeating `{ name: string, age: number }` everywhere, you name it once and reuse the name — very similar to defining a Pydantic model.

Example: `interface User { name: string; age: number; }`

5. How It Works

```
// Basic types
let username: string = "gelan";
let age: number = 21;
let isStudent: boolean = true;
let scores: number[] = [90, 85, 78];

// A typed function
function add(a: number, b: number): number {
  return a + b;
}

// An object shape, defined once and reused
interface User {
  name: string;
  age: number;
  isStudent: boolean;
}

const gelan: User = { name: "Gelan", age: 21, isStudent: true };
```

6. Real-World Example

In a job application form, TypeScript can guarantee at compile time that `submitApplication()` is always called with an object that actually has `email` and `resumeUrl` fields — no more "Cannot read property of undefined" errors discovered by a real user in production.

7. Practical Example

```
interface Product {
  id: number;
  name: string;
  price: number;
}

function formatPrice(product: Product): string {
  return `$$${product.price.toFixed(2)}`;
}

const shoes: Product = { id: 1, name: "Sneakers", price: 49.99 };
formatPrice(shoes); // "$49.99"
```

8. Common Mistakes

Common Mistake

Typing everything as `any` to make errors go away. This silences TypeScript entirely for that value — you lose all the protection you're trying to gain. Treat `any` as a last resort, not a shortcut.

Believing types add runtime safety. TypeScript's checks disappear once your code compiles to plain JavaScript — they only protect you while you're writing the code, not against bad data arriving from a real network request at runtime (that still needs actual validation).

9. "What If?" Section

- What if you pass a string where `formatPrice` expects a `Product`? TypeScript flags it immediately, in your editor, before you run anything.
- What if a field is sometimes missing? Mark it optional with a `?`: `price?: number`; — now TypeScript forces you to handle the missing case.

10. Check Your Understanding

- 1 What problem does TypeScript solve that plain JavaScript doesn't catch?
- 2 What is type inference, and when might you skip writing an explicit type?
- 3 Why is overusing `any` risky?
- 4 Define an interface for a `Task` with `id`, `title`, and `done`.

11. Mini Challenge

Practice

Write an interface `Book` with `title`, `author`, and `pages`, then write a typed function `isLongBook(book: Book): boolean` that returns `true` if `pages > 400`.

12. Lesson Summary

- TypeScript catches type mistakes while you write code, not after you run it.
- Type inference means you don't have to annotate everything explicitly.
- Interfaces name and reuse object shapes, similar to Pydantic models.
- `any` disables type checking — use it sparingly.

13. Can You Explain It?

Goal

1) How is a TypeScript interface similar to a Pydantic model you've used in FastAPI? 2) What's one thing TypeScript can't protect you from, even with perfect types?

Lesson 2.2 — Typing Props & State

1. What You Will Learn

- How to type a component's props with an interface.
- How to type `useState` so React knows what kind of value it holds.
- How TypeScript catches prop-related bugs before you even open the browser.

2. Why This Matters

This lesson is the payoff of Level 1 and Level 2 combined — it's what "React + TypeScript" actually means day to day, and it's most of what you'll type once you start building real components.

3. The Simple Explanation

Every component you wrote in Level 1 quietly accepted *any* props, with no protection. TypeScript lets you declare exactly what props a component expects, so misusing it — a typo in a prop name, a missing required prop, a string where a number was expected — becomes an editor error instead of a silent bug or a broken page.

4. Key Terms

Props interface

Simple definition: An interface describing exactly what props a component accepts.

In plain English: Convention is to name it after the component plus "Props", and pass it to the component's parameter.

Example: `interface UserCardProps { name: string; role: string; }`

Generic

Simple definition: A type that takes another type as a parameter, so it stays reusable.

In plain English: `useState<number>` tells TypeScript exactly what kind of value this particular state holds — you're filling in the blank inside the angle brackets.

Example: `const [count, setCount] = useState<number>(0);`

5. How It Works

```
interface UserCardProps {
  name: string;
  role: string;
  isOnline?: boolean; // optional prop
}

function UserCard({ name, role, isOnline = false }: UserCardProps) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
      <p>{isOnline ? " Online" : " Offline"}</p>
    </div>
  );
}

// Usage – TypeScript will error if `name` or `role` is missing:
<UserCard name="Gelan" role="Engineer" />
```

Typing state usually needs no extra work — TypeScript infers it from the initial value:

```
const [count, setCount] = useState(0); // inferred as number
const [name, setName] = useState(""); // inferred as string

// But when the initial value doesn't reveal the full type, be explicit:
const [user, setUser] = useState<User | null>(null);
// This says: user is EITHER a User object OR null – common for data
// that hasn't loaded yet.
```

6. Real-World Example

A form component that expects an `onSubmit` function prop — if a teammate forgets to pass it, or passes a value that isn't a function, TypeScript catches it immediately instead of the bug surfacing only when a real user clicks submit.

7. Practical Example

```
interface TodoItemProps {
  text: string;
  done: boolean;
  onToggle: () => void;
}

function TodoItem({ text, done, onToggle }: TodoItemProps) {
  return (
    <li onClick={onToggle} style={{ textDecoration: done ? "line-through" : "none" }}>
      {text}
    </li>
  );
}
```

`onToggle: () => void` types a function prop: it takes no arguments and returns nothing useful. This is exactly how you'll type the "tell my parent something happened" functions you'll rely on constantly once you reach Level 3.

8. Common Mistakes

Common Mistake

Forgetting the `?` on genuinely optional props, forcing every caller to pass a value that often has a sensible default.

Typing `useState(null)` with no generic, then being confused why TypeScript won't let you later set it to a `User` object — without `<User | null>`, TypeScript assumes the state can *only* ever be `null`.

9. "What If?" Section

- What if `UserCard` is called with a number for `name`? TypeScript errors immediately, before you ever open the browser.
- What if you want a prop that's only ever one of a few specific strings, like a button's `size`? Use a **union of literals**: `size: "small" | "medium" | "large";`

10. Check Your Understanding

- 1 Why do we usually name a props interface `ComponentNameProps`?
- 2 What does the `?` after a prop name mean?
- 3 Why would you write `useState<User | null>(null)` instead of just `useState(null)`?
- 4 How would you type a prop called `onDelete` that's a function taking no arguments and returning nothing?

11. Mini Challenge

Challenge

Write a fully typed Counter component: it should accept an optional step prop (number, default 1), hold its count in typed state, and expose an onChange: (newCount: number) => void prop that fires whenever the count changes.

12. Lesson Summary

- Define a props interface per component and pass it as the parameter's type.
- TypeScript infers state types from the initial value — be explicit when that value doesn't reveal enough (e.g. null).
- Function props are typed with an arrow-function type signature, e.g. () => void.
- Union types like `User | null` model "this could be one of several shapes."

13. Can You Explain It?

Goal

1) Why does typing props matter more as a codebase grows and more people touch it? 2) What would go wrong at runtime (not compile time) if you skipped typing entirely — and why does TypeScript catching it earlier matter?

Review Session — Levels 0 through 2

Mixed review. Don't just answer the newest material — some of these reach back to Level 0.

Easy

- 1 What does `const` mean, and why is it preferred over `let` by default?
- 2 What is a component?
- 3 What does `useState` return?

Medium

- 1 Rewrite `<UserCard name={props.name} />` using destructured props in the function signature.
- 2 Why does `onClick={doSomething() }` behave differently than `onClick={doSomething}` ?
- 3 Write a typed interface for a props object with a required `title: string` and optional `subtitle`.

Hard

- 1 Explain, end to end, what happens from the moment a user clicks a button wired to `setCount(count + 1)` to the moment the new number appears on screen.
- 2 Why might `useState<number []>( [])` be safer than `useState( [])` in a larger codebase?

Important

What Should I Still Remember?

The difference between props and state, without hesitating.

Why `.map()` shows up constantly in React code.

How to write a props interface and use it to type a component's parameter.

Why `any` quietly defeats the purpose of TypeScript.

Answer Key (Levels 0–2)

Short, direct answers with the reasoning included. If yours doesn't match the reasoning, that's the part to revisit — not just the final word.

Lesson 0.1

Q: Difference between `const` and `let`?

A: `const`'s binding can't be reassigned; `let`'s can. Both are block-scoped, unlike older `var`.

Q: Arrow function for `square(x)`?

A: `const square = (x) => x * x;`

Q: What does destructuring `let` you do?

A: Pull specific values out of an object or array into their own named variables in one line, instead of accessing them one by one.

Q: What does `[1,2,3].map(n => n+1)` return?

A: `[2, 3, 4]` — a new array, original untouched.

Q: Why `async/await` over raw Promises?

A: It reads top-to-bottom like synchronous code, making multi-step asynchronous logic far easier to follow than chained `.then()` calls.

Lesson 1.1

Q: What does 'declarative' mean?

A: You describe the desired end result (what the UI should look like) rather than the step-by-step instructions to reach it.

Q: What is a component?

A: A reusable function that returns UI — the basic building block of a React app.

Q: Why keep a Virtual DOM?

A: So React can compare old vs. new output and update only the real, changed parts of the page — much faster than redrawing everything.

Lesson 1.2

Q: Why no full if statement in {}?

A: Curly braces in JSX only accept expressions (things that evaluate to a value); an if statement doesn't produce a value, so a ternary is used instead.

Q: Why className instead of class?

A: `class` is a reserved word in JavaScript, so JSX can't use it as an attribute name.

Q: What does a Fragment avoid?

A: An unnecessary extra wrapper element (like a div) in the actual rendered HTML, when you just need to group elements for JSX's one-root-element rule.

Lesson 1.3

Q: What are props?

A: Data passed into a component from its parent, similar to function arguments.

Q: Why can't a component reassign its props?

A: Props are owned by the parent; allowing children to change them would break the predictable, one-directional flow of data React relies on.

Q: What is props.children for?

A: It captures whatever JSX is nested between a component's opening and closing tags, letting components act as wrappers.

Lesson 1.4

Q: State vs props?

A: Props come from outside (the parent) and are read-only; state is owned by the component itself and can change over time.

Q: Why does `onClick={fn()}` differ from `onClick={fn}`?

A: `fn()` calls the function immediately during render and passes its return value as the handler; `fn` passes the function itself, to be called later on the actual click.

Q: Why not mutate state directly?

A: React only knows to re-render when you call the setter function; direct mutation changes the value in memory but never tells React to update the page.

Lesson 2.1 & 2.2

Q: What does TypeScript catch that JS doesn't?

A: Type mismatches — wrong argument types, missing object fields, typos in property names — flagged in the editor before the code ever runs.

Q: Why is overusing ``any`` risky?

A: It turns off type checking for that value entirely, silently reintroducing the exact bugs TypeScript is meant to catch.

Q: Why type `useState(null)` explicitly?

A: The initial value (`null`) alone doesn't reveal that the state will later hold a `User` object; without the explicit union type, TypeScript would assume it can only ever be `null`.

Glossary (Levels 0–2)

Term	Definition	Related
Arrow function	A concise function syntax, e.g. (a, b) => a + b.	Functions
async / await	Syntax for writing code that waits on asynchronous operations like data fetching.	Promises
Component	A reusable function that returns UI.	Props, JSX
const / let	Variable declarations; const can't be reassigned, let can.	Variables
Controlled input	A form input whose value is driven by React state.	useState, Events
Declarative UI	Describing the desired result rather than the steps to reach it.	Components
Destructuring	Extracting values from an object/array into named variables in one line.	Props
Event handler	A function that runs in response to a user action like a click.	State
Fragment	A wrapper (<></>) that groups JSX elements without adding an extra <code>div</code> tag.	JSX
Generic	A type parameterized by another type, e.g. useState<number>.	TypeScript
Hook	A function (starting with 'use') that connects a component to React features.	useState
Interface	A named description of an object's shape in TypeScript.	Types, Props
JSX	HTML-like syntax inside JavaScript that compiles to function calls.	Component
Props	Data passed into a component from its parent.	Component, Interface
Props interface	An interface that defines exactly what props a component accepts.	Props, TypeScript
State	Data a component owns that can change over time, triggering re-renders.	useState
Type inference	TypeScript automatically determining a value's type without an explicit annotation.	TypeScript
Union type	A type that can be one of several specified types, e.g. User null.	TypeScript
useState	The hook that gives a component a piece of changeable state.	Hook, State
Virtual DOM	React's in-memory copy of the page used to compute minimal real updates.	Component

Cheat Sheet — Levels 0–2

One page to review the whole course quickly before moving to Part 2.

JavaScript Essentials

```
const x = 5; // can't reassign
let y = 10; // can reassign
const add = (a, b) => a + b; // arrow function
const { name, age } = user; // destructuring
arr.map(item => item * 2); // transform every item
`Hello, ${name}!` // template literal
```

JSX Rules

- One root element per return (or use a Fragment: `<>...</>`).
- `{ }` holds JavaScript expressions only, never statements.
- `className`, not `class`.

Component & Props Pattern

```
interface CardProps {
  title: string;
  subtitle?: string;
}

function Card({ title, subtitle }: CardProps) {
  return <div><h2>{title}</h2><p>{subtitle}</p></div>;
}
```

State Pattern

```
const [count, setCount] = useState<number>(0);
const [user, setUser] = useState<User | null>(null);
<button onClick={() => setCount(count + 1)}>+1</button>
```

Frequent Mistakes to Check For

- `onClick={fn()}` instead of `onClick={fn}`.
- Mutating state directly instead of calling the setter.
- Typing something any just to silence an error.
- Missing `?` on a genuinely optional prop.

What's Next

You now have a real foundation: JavaScript essentials, JSX, components, props, state, and typing all of it with TypeScript. That's enough to build small but genuinely interactive, typed components.

Part 2 will cover (Levels 3–4)

- Rendering lists and handling keys correctly.
- Conditional rendering patterns beyond simple ternaries.
- Forms in depth — validation, multiple fields, submission.
- `useEffect` and fetching real data from an API (a natural fit with your FastAPI backend work).
- A set of debugging exercises — deliberately broken components to fix.

Part 3 will cover (Levels 5–7)

- Custom hooks, Context, and generics for more advanced typing.
- Three mini projects combining everything.
- The Final Challenge project and a full exam with a separate answer key.

Goal

Before requesting Part 2: rebuild the `Counter` and `TodoItem` examples from Lesson 1.4 and 2.2 from memory, with full types. If you can do that cleanly, you're genuinely ready to move forward — not just familiar with the words.